Your First Shiny App

Shiny: the reader picks the county and the years

Eric M. Delmelle · Lehigh University

Today

Today you build **Milestone M2**. Reading: **Moraga Ch. 13–14**.

> **M2 — Reactive app** Shiny reactivity: the user selects a region and a time period — a dropdown, a slider, or an animation — and the maps and summaries respond.

Your M1 dashboard shows all 21 years and the one county you chose. M2 hands both choices to the reader. Two small apps first, to see how Shiny thinks; then the M1 dashboard, with a dropdown and a slider.

M2 asks for	Where it is today
Shiny reactivity	§1–§2
a region and a time period	a dropdown and a slider, §3; or a play button, §4
the maps and summaries respond	§3

Nothing new to install: `shiny` came with `flexdashboard`.

Your first app

A `flexdashboard` is knitted once: R draws the panels, writes an HTML file, and stops. With Shiny, R keeps running behind the page. The reader moves a slider, R reruns the code that uses it, and the panel is redrawn.

Moraga's first example (§13.6.1): a box for a sample size, and a histogram of that many normal draws. An app is a `ui`, the page; a `server`, the R code; and `shinyApp()`, which joins them.

```
library(shiny)

ui <- fluidPage(
  numericInput(inputId = "n", label = "Sample size", value = 25),
  plotOutput(outputId = "hist")
)
```

```
server <- function(input, output) {
  output$hist <- renderPlot({
    hist(rnorm(input$n))
  })
}

shinyApp(ui = ui, server = server)
```

Run it. Copy the whole script into a new R script, select all (**Ctrl+A**) and run (**Ctrl+Enter**). The app opens in a window; type 500 in the box. Stop it with the red stop sign in the Console. A piece alone, say the `ui`, only creates an object, and nothing opens.

> **!** Three rules
>
> 1. Every input has an id; its value is `input$id`.
> 2. Every output has an id: `output$id <- render*({ ... })` in the server draws it, `*Output("id")` in the ui places it.
> 3. `input$...` can only be read inside a `render*()` or a `reactive()`.

Try it. Replace the box with a slider from 10 to 1,000, and add the mean of the sample under the histogram: `textOutput("mean")` in the ui, `output$mean <- renderText({ ... })` in the server.

```
library(shiny)

ui <- fluidPage(
  sliderInput("n", "Sample size", min = 10, max = 1000, value = 100),
  plotOutput("hist"),
  textOutput("mean")
)

server <- function(input, output) {
  output$hist <- renderPlot({
    hist(rnorm(input$n))
  })
  output$mean <- renderText({
    paste("Mean:", round(mean(rnorm(input$n)), 3))
  })
}

shinyApp(ui, server)
```

How reactivity flows

Run the solution with the slider at 10. The mean does not match the histogram: each `render*()` calls `rnorm()` on its own, so the two outputs show two different samples. The fix is a `reactive()`:

draw the sample once, and let both outputs read it.

```
library(shiny)

ui <- fluidPage(
  sliderInput("n", "Sample size", min = 10, max = 1000, value = 100),
  selectInput("col", "Colour", choices = c("grey", "steelblue", "firebrick")),
  plotOutput("hist"),
  textOutput("mean")
)

server <- function(input, output) {
  x <- reactive(rnorm(input$n))              # one sample, shared

  output$hist <- renderPlot({
    hist(x(), col = input$col, main = NULL)
  })
  output$mean <- renderText({
    paste("Mean of this sample:", round(mean(x()), 3))
  })
}

shinyApp(ui, server)
```

Move the slider: a new sample, and both outputs change. Change the colour: same bars, same mean. Shiny reruns only the code that reads the input you changed.

A `reactive()` is a spreadsheet cell with a formula: it recomputes when the cells it reads change. Read it with parentheses: `x()`.

⚠ Two mistakes everyone makes once

You wrote	You get	Fix
`x <- rnorm(input$n)`, directly in `server`	*Can't access reactive value 'n' outside of reactive consumer*	`x <- reactive(rnorm(input$n))`
`hist(x)`	*'x' must be numeric*, or an error naming `reactive`	`hist(x())`

From M1 to M2: the dashboard

Save a copy of `dashboard.Rmd` as **`dashboard_app.Rmd`** and work on the copy; M1 stays online as it is. The changes:

In M1	In M2
the YAML header	plus `runtime: shiny`
`focus <- "Cuyahoga"`	a dropdown, `input$county`, in a sidebar
all years	a slider, `input$years`, in the same sidebar
`by_county`, computed once	`by_county()`, a `reactive()` that follows the slider
`leaflet(...)`, `ggplotly(p)`, `datatable(...)`	the same, inside `renderLeaflet({ })`, `renderPlotly({ })`, `renderDT({ })`

The new screen has one more column, on the left, for the inputs:

In a dashboard there is no `ui`, `server` or `shinyApp()`: the layout is the ui, and a `render*()` in a panel is enough. Code that reads `input$` goes in a `reactive()` or a `render*()`; code that does not stays in the setup chunk.

The whole file, with `# M2` on what changed:

````
---
title: "Lung cancer in Ohio, 1968--1988"
output:
  flexdashboard::flex_dashboard:
    orientation: columns
    vertical_layout: fill
    theme: cosmo
    source_code: embed
runtime: shiny                                  # M2
---

```{r setup, include=FALSE}
library(flexdashboard)
library(shiny)                                  # M2
library(sf)
library(dplyr)
library(readr)
library(ggplot2)
library(plotly)
library(leaflet)
library(DT)
library(tigris)
options(tigris_use_cache = TRUE)
knitr::opts_chunk$set(echo = FALSE, warning = FALSE, message = FALSE)

# data: one row per county and year
oh <- read_csv("data/ohio_lung_cancer_1968_1988.csv",
               col_types = cols(GEOID = "c"))          # GEOID as text

# boundaries
````
````

```
oh_map <- counties(state = "OH", cb = TRUE, year = 2020,
                   progress_bar = FALSE) |>
  st_transform(4326)                              # leaflet wants lon/lat

# M2: Ohio by year reads no input, computed once
ohio_year <- oh |>
  group_by(year) |>
  summarise(Y = sum(Y), E = sum(E), .groups = "drop") |>
  mutate(SIR = Y / E, area = "Ohio")
```

Column {.sidebar data-width=250}

```{r inputs}
# M2: the reader's choices
selectInput("county", "County",
            choices = sort(unique(oh$county)), selected = "Cuyahoga")

sliderInput("years", "Years",
            min = min(oh$year), max = max(oh$year),
            value = range(oh$year), step = 1, sep = "")   # sep: 1968, not 1,968
```

Pick a county, then drag either end of the slider. The map, the time
series and the table follow.

```{r reactive}
# M2: SIR by county, for the selected years
by_county <- reactive({
  oh |>
    filter(between(year, input$years[1], input$years[2])) |>
    group_by(GEOID, county) |>
    summarise(Y = sum(Y), E = sum(E), .groups = "drop") |>
    mutate(SIR = Y / E)
})
```

Column {data-width=600}

SIR by county, selected years (black outline: selected county)

```{r map}
breaks <- c(0, 0.5, 2/3, 0.9, 1/0.9, 1.5, 2, Inf)  # Exercise 4 classes

renderLeaflet({                                    # M2
```

```
  d <- left_join(oh_map, by_county(), by = "GEOID")    # M2: join inside
  pal <- colorBin(rev(RColorBrewer::brewer.pal(7, "RdBu")),
                  domain = d$SIR, bins = breaks)

  leaflet(d) |>
    addProviderTiles(
      providers$Esri.WorldImagery,
      options = providerTileOptions(opacity = 0.5)
    ) |>
    addPolygons(
      fillColor = ~pal(SIR), fillOpacity = 0.8,
      color = "white", weight = 1,
      label = ~sprintf("%s: SIR %.2f (%.0f observed, %.0f expected)",
                       NAME, SIR, Y, E),
      highlightOptions = highlightOptions(weight = 2, color = "black",
                                          bringToFront = TRUE)
    ) |>
    addPolygons(data = filter(d, NAME == input$county),   # M2: outline
                fill = FALSE, color = "black", weight = 3) |>
    addLegend("bottomright", pal = pal, values = ~SIR,
              title = paste0("SIR<br>", input$years[1], "-", input$years[2]),
              labFormat = labelFormat(digits = 2), opacity = 0.8)
})
```

> SIR = observed / expected over the selected years. Red: more cases than
> expected; blue: fewer.

Column {data-width=400}

SIR by year (grey band: selected years; dashed line: as many cases as expected)

```{r timeseries}
renderPlotly({                                      # M2
  one <- oh |>
    filter(county == input$county) |>               # M2: was focus
    mutate(SIR = Y / E, area = input$county)

  cols <- setNames(c("black", "#B2182B"), c("Ohio", input$county))

  p <- ggplot(bind_rows(ohio_year, one), aes(year, SIR, colour = area)) +
    annotate("rect", xmin = input$years[1] - 0.5,
             xmax = input$years[2] + 0.5,
             ymin = -Inf, ymax = Inf, fill = "grey90") +  # M2: selected years
    geom_hline(yintercept = 1, linetype = 2, colour = "grey50") +
    geom_line(linewidth = 0.8) +
```

```
    geom_point(size = 1.2) +
    scale_colour_manual(values = cols) +
    labs(x = NULL, y = "SIR", colour = NULL) +
    theme_minimal()

  ggplotly(p)                                    # hover, zoom
})
```

Counties, selected years

```{r table}
renderDT({                                         # M2
  by_county() |>
    transmute(County = county, Observed = Y,
              Expected = round(E, 1), SIR = round(SIR, 2)) |>
    arrange(desc(SIR)) |>
    datatable(rownames = FALSE, options = list(pageLength = 10)) |>
    formatStyle("County", target = "row",
                fontWeight = styleEqual(input$county, "bold"))  # M2: bold row
})
```

> Data: Moraga (2019), Ch. 7. Boundaries: US Census Bureau, via `tigris`.

Run it. With `runtime: shiny`, RStudio's **Knit** button becomes **Run Document**: click it, or run `rmarkdown::run("dashboard_app.Rmd")`. Knitting or `quarto render` cannot work: a Shiny dashboard is a running R session, not a file. GitHub Pages cannot run R either, so M2 is demonstrated live from your laptop.

Pick another county: only the outline moves, since the colours depend on the years. Then compare Cuyahoga over 1968–1972 and 1984–1988: its SIR moves from about 0.8 to about 1.4, a change the 21-year map of M1 averages away.

A play button (optional)

M2 accepts an animation in place of the range slider. Replace the `sliderInput()` with:

```
sliderInput("year", "Year",
            min = min(oh$year), max = max(oh$year),
            value = min(oh$year), step = 1, sep = "",
            animate = animationOptions(interval = 1500))   # play button
```

Then, with **Edit > Replace**, change `input$years[1]` and `input$years[2]` to `input$year`. Press play: small counties flicker between red and blue from year to year, the small-numbers problem of Exercise 4 in motion.

Your turn: from Ohio to your data

In the example	For your data
`county`, in the dropdown, `group_by()` and the time series	your area-name column
`NAME == input$county` on the map	the name column of your boundaries
`year`, in the slider and `between()`	your time column; `sliderInput()` takes `Date`s too
`Y`, `E`	your observed and expected counts, or a rate as in M1

Before you demonstrate M2

1. **Session > Restart R**, then **Run Document**.
2. Another county: the time series and the outline follow.
3. Another period: the colours, the legend and the table follow.
4. Run it once before class, with internet, so `tigris` caches the boundaries.

When it breaks

What you see	Likely cause	Fix
No **Run Document** button	`runtime: shiny` missing, or indented	at the left margin of the YAML header
A page where nothing responds	an old `.html` file, opened directly	**Run Document**
A panel shows an error in red	that panel's code failed	read the message; check `()` after `by_county`
Each change takes seconds	heavy work inside a `render*()`	move it to the setup chunk
`quarto render` fails on `dashboard_app.Rmd`	Quarto tries to render the `.Rmd`	`render: - "*.qmd"` in `_quarto.yml`

Next

Next week the reader brings the data: `fileInput()`, a shapefile and a `dygraphs` time series (Moraga Ch. 15).

References

Chang, W., Cheng, J., Allaire, J. J., Sievert, C., Schloerke, B., Xie, Y., Allen, J., McPherson, J., Dipert, A., & Borges, B. (2024). *shiny: Web application framework for R*. https://shiny.posit.co/

Moraga, P. (2019). *Geospatial health data: Modeling and visualization with R-INLA and Shiny*. Chapman & Hall/CRC, Ch. 13 and 14.

Wickham, H. (2021). *Mastering Shiny*. O'Reilly. Ch. 2–3. https://mastering-shiny.org/